



Mini Project Report – 18

Bachelor of Computer Application –AI and ML

Semester - IV

Sub: Full Stack Development

Topic: Student Management System API

By

Name: Aryan Sovit

Reg no.: 2411021240074

Faculty Name: VEERA RAGHAV K

Faculty Signature: _____

Department of Computer Application

Alliance University

Chandapura - Anekal Main Road, Anekal

Bengaluru - 562 106

March 2026

Introduction

In modern web applications, **APIs (Application Programming Interfaces)** are essential for enabling communication between different software systems. APIs

allow applications to exchange data in a structured format, usually **JSON (JavaScript Object Notation)**. This ensures that data is transferred efficiently and can be easily understood by both developers and machines.

The **Student Management System API (SMS API)** is a backend application developed to manage student records effectively. It is built using **Node.js** as the runtime environment and **Express.js** as the framework for creating server-side applications. The system uses **MongoDB**, a NoSQL database, to store and organize student information securely.

The system provides the following functionalities:

- **Create** – Add new student records to the database.
- **Read** – View existing student information.
- **Update** – Modify student details as required.
- **Delete** – Remove student records that are no longer needed.

The API is tested using **Postman**, which allows developers to send **GET, POST, PUT, and DELETE** requests and verify responses. The database is managed using **MongoDB Compass**, which provides an easy-to-use interface for viewing and organizing data.

This project helps in understanding:

- RESTful API development and design principles.
- Backend server setup using Node.js and Express.js.
- Database connectivity and management with MongoDB.
- Implementation of CRUD operations.
- Testing and verification of APIs.

Objectives

The main objectives of this project are:

- To develop a **RESTful API** using Node.js and Express.js.
- To connect the application with a **MongoDB database**.
- To implement **CRUD operations** for managing student records.
- To test and verify API endpoints using **Postman**.
- To understand the **backend development workflow**.
- To store, retrieve, update, and delete student data efficiently.

Tools and Technologies Used

Tool	Purpose
Node.js	Used as the runtime environment to run backend JavaScript code

Express.js	A web framework used to create APIs and handle server requests
MongoDB	A NoSQL database used to store and manage student data
MongoDB Compass	Graphical interface used to view and manage MongoDB databases
VS Code	Source code editor used for writing and editing the project code
Postman	Tool used to test API requests like GET, POST, PUT, and DELETE
CMD / Terminal	Used to run server commands such as starting the Node.js server

Project Setup (Commands Used)

The following commands were executed in Command Prompt (CMD) to set up the project.

Step 1: Create Project Folder

```
mkdir sms-api cd
sms-api
```

Step 2: Initialize Node Project

```
npm init -y
```

Step 3: Install Required Packages

```
npm install express mongoose body-parser
```

MongoDB Database Setup

MongoDB is used as the database to store student records.

Default MongoDB Connection URL

```
mongodb://127.0.0.1:27017/studentdb
```

Database Creation

When the server runs for the first time, MongoDB automatically creates the database named:

```
studentdb
```

Inside this database a collection called:

students

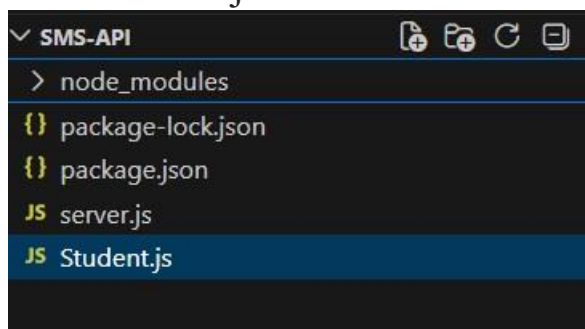
is created automatically when data is inserted.

MongoDB Compass can be used to visually see the stored records.

Project Structure

The project structure inside VS Code is organized as follows: sms-api

```
|
|— node_modules
|— package-lock.json
|— package.json
|— server.js
|— Student.js
```



Explanation:

- Student.js → defines student structure
 - server.js → main server file
 - package.json → project dependencies
-

Student Schema (Student.js Code):

```
const mongoose = require("mongoose"); const
StudentSchema = new mongoose.Schema({
  name: String, age:
  Number, course:
  String
});
module.exports = mongoose.model("Student",StudentSchema);
```

Server Implementation (server.js Code):

```
const mongoose = require("mongoose");
const express = require("express"); const
Student = require("./Student") const app
= express();
app.use(express.json());
```

```
// Mongo DB CONNECTION
mongoose.connect("mongodb://127.0.0.1:27017/studentdb")
.then(()=>console.log("MongoDB Connected"))
.catch(err=>console.log(err));
```

```
// ADD STUDENT ----- > POST METHOD
```

```
app.post("/students",async(req,res)=>{
const student = new Student(req.body);
const result = await student.save();
res.json(student); });
```

```
// GET STUDENTS
```

```
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});
```

```
//Update Students -----> Put Method
app.put("/students/:id",async(req,res)=>{ const
student = await Student.findByIdAndUpdate(
  req.params.id,
  req.body,
  {new:true}
) ;
res.send(student);
});
```

```
//Delete STUDENTS
```

```
app.delete("/students/:id",async (req,res)=>{
  const student = await Student.findByIdAndDelete(req.params.id);
  res.send(student);
```

```
});
```

// Creating Local host server

```
app.listen(3000,()=>{  
  console.log("http://localhost:3000/")  
});
```

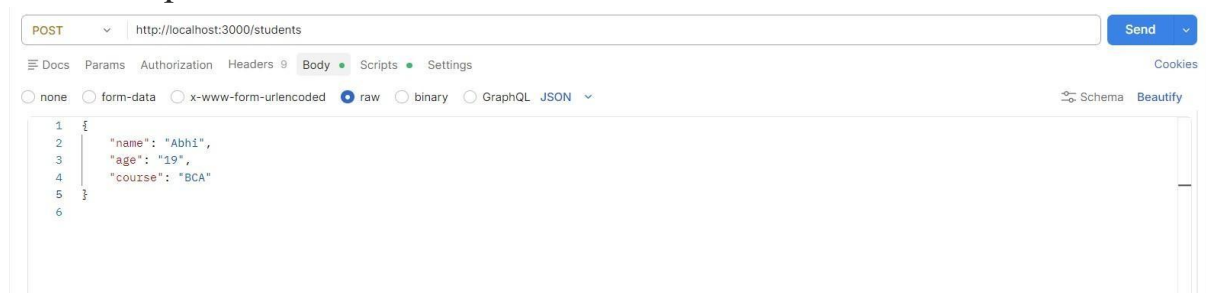
Running the Server in Terminal(node server.js):

```
PS C:\Users\Lakha\bca 4\semester 4\Full Stack Development\sms-api> node server.js  
http://localhost:3000/  
MongoDB Connected  
□
```

API Testing using Postman:

Postman is used to test the API endpoints.

POST Request:



GET Request:



DELETE Request:

DELETE Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

```
1 {
2   "name": "Ashu",
3   "age": 19,
4   "course": "BCA"
5 }
6
```

Body Cookies Headers 7 Test Results 1/1

200 OK • 18 ms • 315 B Save Response

{ JSON Preview Visualize

```
1 {
2   "_id": "69b2ea39dcfd5b5ad3f12be7",
3   "name": "Ansh",
4   "age": 19,
5   "course": "BCA",
6   "__v": 0
7 }
```

PUT Request:

PUT Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

```
1 {
2   "name": "Ashu",
3   "age": 19,
4   "course": "BCA"
5 }
6
```

Body Cookies Headers 7 Test Results 1/1

200 OK • 21 ms • 315 B Save Response

{ JSON Preview Visualize

```
1 {
2   "_id": "69b2ea47dcfd5b5ad3f12be9",
3   "name": "Ashu",
4   "age": 19,
5   "course": "BCA",
6   "__v": 0
7 }
```

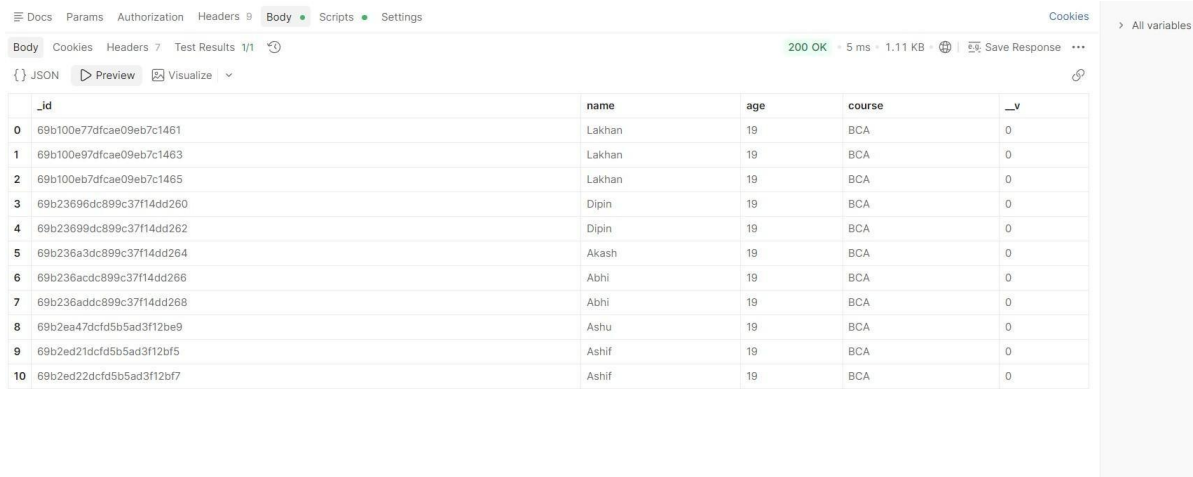
Browser Output:

localhost:3000/students

pretty-print

```
{ "_id": "69b108e7dfcae09eb7c1461", "name": "Lakhan", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b108e97dfcae09eb7c1463", "name": "Lakhan", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b108e97dfcae09eb7c1465", "name": "Lakhan", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b23696dc899c37f14dd260", "name": "Dipin", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b23696dc899c37f14dd262", "name": "Dipin", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b236a3dc899c37f14dd264", "name": "Akash", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b236acdc899c37f14dd266", "name": "Abhi", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b236addc899c37f14dd268", "name": "Abhi", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b2ea47dcfd5b5ad3f12be9", "name": "Ashu", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b2ed21dcfd5b5ad3f12bf5", "name": "Ashif", "age": 19, "course": "BCA", "__v": 0 }, { "_id": "69b2ed22dcfd5b5ad3f12bf7", "name": "Ashif", "age": 19, "course": "BCA", "__v": 0 } ]
```

Students API response (JSON File output that you will GET to see after Posting all the 10 students details):



The screenshot shows a REST client interface with a response status of 200 OK. The response body is a JSON array of 10 student objects. The table below represents the data shown in the screenshot.

	_id	name	age	course	_v
0	69b100e77dfcae09eb7c1461	Lakhan	19	BCA	0
1	69b100e97dfcae09eb7c1463	Lakhan	19	BCA	0
2	69b100eb7dfcae09eb7c1465	Lakhan	19	BCA	0
3	69b23696dc899c37f14dd260	Dipin	19	BCA	0
4	69b23699dc899c37f14dd262	Dipin	19	BCA	0
5	69b236a3dc899c37f14dd264	Akash	19	BCA	0
6	69b236acdc899c37f14dd266	Abhi	19	BCA	0
7	69b236addc899c37f14dd268	Abhi	19	BCA	0
8	69b2ea47dcfd5b5ad3f12be9	Ashu	19	BCA	0
9	69b2ed21dcfd5b5ad3f12bf5	Ashif	19	BCA	0
10	69b2ed22dcfd5b5ad3f12bf7	Ashif	19	BCA	0

```
[
  {
    "_id": "69b100e77dfcae09eb7c1461",
    "name": "Ashif",
    "age": 19,
    "course": "BCA",
    " __ ": 0
  },
  {
    "_id": "69b100e97dfcae09eb7c1463",
    "name": "Ashif",
    "age": 19,
    "course": "BCA",
    " __ ": 0
  },
  {
    "_id": "69b100eb7dfcae09eb7c1465",
    "name": "Lakhan",
```



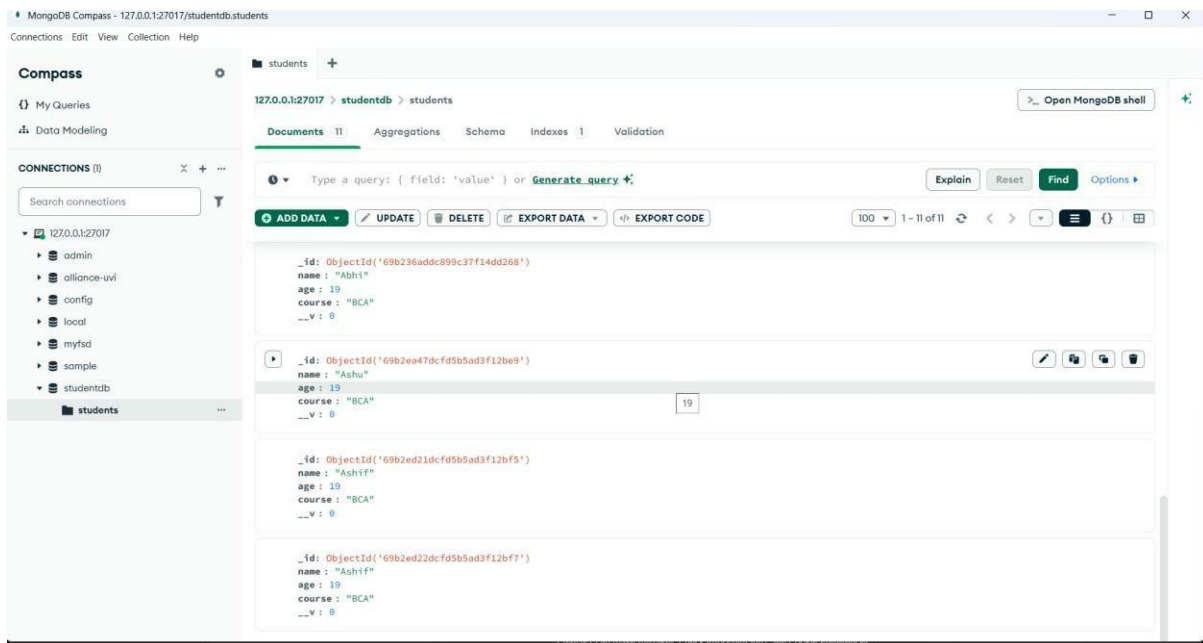
```
"age": 19,
"course": "BCA",
"__": 0
},
{
  "_id": "69b23696dc899c37f14dd260",
  "name": "Dipin",
  "age": 19,
  "course": "BCA",
  "__": 0
},
{
  "_id": "69b23699dc899c37f14dd262",
  "name": "Dipin",
  "age": 19,
  "course": "BCA",
  "__": 0
},
{
  "_id": "69b236a3dc899c37f14dd264",
  "name": "Akash",
  "age": 19,
  "course": "BCA",
  "__": 0
},
{
  "_id": "69b236acdc899c37f14dd266",
  "name": "Abhi",
  "age": 19,
```

```
"course": "BCA",
"__": 0
},
{
  "_id": "69b236addc899c37f14dd268",
  "name": "Abhi",
  "age": 19,
  "course": "BCA",
  "__": 0
},
{
  "_id": "69b2ea47dcfd5b5ad3f12be9",
  "name": "Ashu",
  "age": 19,
  "course": "BCA",
  "__": 0
},
{
  "_id": "69b2ed21dcfd5b5ad3f12bf5",
  "name": "Lakhan",
  "age": 19,
  "course": "BCA",
  "__": 0
},
{
  "_id": "69b2ed22dcfd5b5ad3f12bf7",
  "name": "Lakhan",
  "age": 19,
  "course": "BCA",
```

```
"__": 0
}
]
```

MongoDB Compass Database View:

MongoDB Compass shows stored student records in the studentdb database inside the students collection.



Results / Output

The Student Management System API was successfully developed using Node.js, Express.js, and MongoDB.

The API allows users to perform CRUD operations:

- Add new students
- Retrieve student records
- Update student details
- Delete student records

All API endpoints were successfully tested using Postman.

The student data was stored and retrieved correctly from MongoDB.

Conclusion

This mini project helped in understanding the implementation of REST APIs using Node.js and MongoDB. The project demonstrated how backend systems handle data storage, retrieval, and manipulation using CRUD operations.

Through this project, practical experience was gained in:

- Backend API development
- Database integration
- API testing
- Server deployment basics

The system can be further enhanced by adding authentication, validation, and frontend interface

